

Package ‘phylopomp’

April 3, 2026

Contents

as.data.frame	2
curtail	2
diagram	3
gendat	4
geneal	5
geneal_sum	5
genealogy diagram internals	6
getInfo	8
lbdp	10
lineages	11
moran	13
newick	14
parse_newick	15
phylopomp	17
plot	17
reexports	20
s2i2r2	21
seir	23
si2r	26
siir	28
simulate	29
sir	31
strains	33
twospecies	35
twoundead	38
yaml	40

Index	41
--------------	-----------

as.data.frame	<i>Coerce to a Data Frame</i>
---------------	-------------------------------

Description

Functions to coerce an object to a data frame.

Usage

```
## S3 method for class 'gplin'
as.data.frame(x, ...)
```

Arguments

x	any R object.
...	additional arguments to be passed to or from methods.

Details

An object of class ‘gplin’ is coerced to a data frame by means of `as.data.frame`.

curtail	<i>Curtail a genealogy to the given time</i>
---------	--

Description

Discards all nodes beyond the given time.

Usage

```
curtail(object, time = NA, t0 = NA)
```

Arguments

object	gpsim object.
t0, time	numeric scalars; determine the time interval for curtailed genealogy

Value

a genealogy object: the curtailed version of the input genealogy.

Examples

```
library(ggplot2)

simulate("SIIR",time=5) -> x

plot_grid(
  x |>
    plot(prune=FALSE,points=TRUE),
  x |>
    curtail(time=3) |>
    plot(prune=FALSE,points=TRUE)+
    expand_limits(x=5),
  ncol=1,align="h",axis="tblr"
)

plot_grid(
  x |>
    plot(prune=TRUE,points=TRUE)+
    geom_vline(xintercept=3),
  x |> curtail(time=3) |>
    plot(prune=TRUE,points=TRUE)+
    geom_vline(xintercept=3)+
    expand_limits(x=5),
  ncol=1,align="h",axis="tblr"
)
```

diagram

*Genealogy process diagram***Description**

Produces a diagram of the genealogy process state.

Usage

```
diagram(
  object,
  prune = TRUE,
  obscure = TRUE,
  m = NULL,
  n = NULL,
  ...,
  digits = 1,
  palette = scales::hue_pal(l = 80, c = 20, h = c(220, 580))
)

## S3 method for class 'gpdiag'
print(x, newpage = is.null(vp), vp = NULL, ...)
```

Arguments

object	gpsim object.
prune	logical; prune the genealogy?
obscure	logical; obscure the demes?
m	width of plotting window, in nodes. By default, the nodes will be adjusted in width to fit the window.
n	height of the pockets, in balls. By default, the balls will be adjusted in size to fit the space available.
...	other arguments, ignored.
digits	non-negative integer; number of decimal digits to print in the node time
palette	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.
x	object of class 'gpdiag'
newpage	draw new empty page first?
vp	viewport to draw plot in

Value

A **grid** graphics object (grob), invisibly.

Examples

```
runSIR(Beta=3,gamma=0.1,psi=0.2,S0=100,I0=5,R0=0,time=2,t0=0) -> x
plot(x,points=TRUE,prune=FALSE)
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))
```

gendat	<i>Genealogy as a data frame</i>
--------	----------------------------------

Description

Converts a given genealogy to a data frame.

Usage

```
gendat(object, obscure = TRUE)
```

Arguments

object	a 'gpgen' object.
obscure	logical; obscure the demes?

Value

A list of objects containing the information pertinent for filtering.

geneal	<i>Bare genealogy</i>
--------	-----------------------

Description

Extracts the bare genealogy from a Markov genealogy process simulation

Usage

```
geneal(object)
```

Arguments

object a ‘gpger’ object, possibly with ‘model’ attribute.

Value

A bare genealogy object.

geneal_sum	<i>Combine genealogies</i>
------------	----------------------------

Description

A summing operator for genealogies.

Usage

```
## S3 method for class 'gpger'
x + y

geneal_sum(...)
```

Arguments

x, y, ... genealogies or lists of genealogies to be summed.

Details

The sum of genealogies is defined to be the union of all the lineages, restricted to the common time-interval, i.e., the intersection of all the individual time-intervals.

Value

combined genealogy.

Examples

```
library(ggplot2)

runSEIR(t0=-1,time=2,E0=0,I0=3) -> x
runSEIR(t0=0,time=3,E0=2,I0=0) -> y
runSEIR(t0=1,time=4,E0=3,I0=2) -> z

plot_grid(
  plot_grid(
    plotlist=lapply(
      list(x,y,z),
      plot,
      points=TRUE,ladderize=FALSE,
      legend.position="none"
    ) |>
    lapply(\(.).+geom_vline(xintercept=c(-1,0,1,2,3,4))),
    ncol=1,align="v",axis="bltr"
  ),
  plot_grid(
    plotlist=lapply(
      list(x+y,y+z,x+y+z),
      plot,
      points=TRUE,ladderize=FALSE,
      legend.position="none"
    ) |>
    lapply(\(.).+geom_vline(xintercept=c(-1,0,1,2,3,4))),
    ncol=1,align="h",axis="bltr"
  )+
  geom_vline(xintercept=c(0,1,2,3,4)),
  nrow=1
)
```

genealogy diagram internals

Diagramming internals

Description

Facilities to produce diagrammatic representations of genealogy process states.

Usage

```
genealogyGrob(object, m = NULL, n = NULL, vp = NULL, palette, ...)
```

```
nodeGrob(object, digits = 1, palette, n = NULL, vp = NULL)
```

```

pocketGrob(object, n, vp = NULL)

ballGrob(object, vp = NULL)

resizingTextGrob(..., vp = NULL)

## S3 method for class 'resizingTextGrob'
drawDetails(x, recording = TRUE)

## S3 method for class 'resizingTextGrob'
preDrawDetails(x)

## S3 method for class 'resizingTextGrob'
postDrawDetails(x)

## S3 method for class 'ballGrob'
drawDetails(x, recording = TRUE)

## S3 method for class 'ballGrob'
preDrawDetails(x)

## S3 method for class 'ballGrob'
postDrawDetails(x)

## S3 method for class 'gpsim'
print(x, ...)

## S3 method for class 'gpgen'
print(x, ...)

## S3 method for class 'gpyaml'
print(x, ...)

```

Arguments

<code>object</code>	list; pocket structure
<code>m</code>	width of plotting window, in nodes. By default, the nodes will be adjusted in width to fit the window.
<code>n</code>	length of longest genealogy
<code>vp</code>	viewport to draw plot in
<code>palette</code>	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.
<code>...</code>	arguments to be passed to textGrob .
<code>digits</code>	non-negative integer; number of decimal digits to print in the node time

<code>x</code>	an object of class ‘grob’ or NULL
<code>recording</code>	A logical value indicating whether a grob is being added to the display list or redrawn from the display list.

Details

Code for the resizing text adapted from a blog post by Mark Heckmann (<https://ryouready.wordpress.com/2012/08/01/creating-a-text-grob-that-automatically-adjusts-to-viewport-size/>).

<code>getInfo</code>	<i>getInfo</i>
----------------------	----------------

Description

Retrieve information from genealogy process simulation

Usage

```
getInfo(
  object,
  prune = TRUE,
  obscure = TRUE,
  extended = TRUE,
  t0 = FALSE,
  time = FALSE,
  structure = FALSE,
  yaml = FALSE,
  ndeme = FALSE,
  lineages = FALSE,
  newick = FALSE,
  nsample = FALSE,
  nroot = FALSE,
  genealogy = FALSE,
  gendat = FALSE
)
```

Arguments

<code>object</code>	gpsim object.
<code>prune</code>	logical; prune the genealogy?
<code>obscure</code>	logical; obscure the demes?
<code>extended</code>	logical; return extended-Newick format?
<code>t0</code>	logical; return the zero-time?
<code>time</code>	logical; return the current time?
<code>structure</code>	logical; return the structure in R list format?

yaml	logical; return the structure in YAML format?
ndeme	logical; return the number of demes?
lineages	logical; return the lineage-count function?
newick	logical; return a Newick-format description of the tree?
nsample	logical; return the number of samples?
nroot	logical; return the number of roots?
genealogy	logical; return the lineage-traced genealogy?
gendat	logical; return the data-frame format?

Value

A list containing the requested elements, including any or all of:

t0 the initial time (a numeric scalar)
time the final time (a numeric scalar)
ndeme the number of demes (an integer)
nsample the number of samples (an integer)
nroot the number of roots (an integer)
newick the genealogical tree, in Newick format (extended-Newick if extended=TRUE)
yaml the state of the genealogy process in YAML format
structure the state of the genealogy process in R list format
lineages a **tibble** containing the lineage count function through time
gendat a **tibble** containing the (obscured) genealogy in a data-frame format
genealogy the lineage-traced genealogy (as a raw vector)

Examples

```
simulate("SIIR",time=3,psi1=1,psi2=0) |>
  simulate(Beta1=2,gamma=2,time=10,psi1=10,psi2=1) |>
  plot()

runSIIR(Beta1=10,Beta2=8,
  S0=200,I1_0=10,I2_0=8,R0=0,time=0,t0=-1) |>
  simulate(psi1=10,time=2) |>
  plot(points=TRUE,obscure=FALSE)

simulate("SIIR",Beta1=2,Beta2=50,gamma=1,psi1=2,
  S0=300,I1_0=20,I2_0=2,time=5) |>
  lineages() |>
  plot()
```

lbdp

*Linear birth-death-sampling model***Description**

The genealogy process induced by a simple linear birth-death process with constant-rate sampling.

Usage

```
runLBDP(time, t0 = 0, lambda = 2, mu = 1, psi = 1, chi = 0, n0 = 5)

continueLBDP(object, time, lambda = NA, mu = NA, psi = NA, chi = NA)

lbdp_pomp(x, lambda, mu, psi, chi = 0, n0 = 1)

lbdp_exact(x, lambda, mu, psi, chi = 0, n0 = 1)
```

Arguments

time	final time
t0	initial time
lambda	per capita birth rate
mu	per capita death rate
psi	per capita non-destructive sampling rate
chi	per capita destructive sampling rate
n0	population size at time t0
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format (i.e., an object that inherits from ‘gpgen’).

Details

`lbdp_pomp` constructs a **pomp** object containing a given set of data and a linear birth-death-sampling process.

`lbdp_exact` gives the exact log likelihood of a linear birth-death process with (optionally destructive) sampling, conditioned on the population size at time 0.

Value

`runLBDP` and `continueLBDP` return objects of class ‘gpsim’ with ‘model’ attribute “LBDP”.

`lbdp_exact` returns the log likelihood of the genealogy. Note that the time since the most recent sample is informative.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:10.1016/j.tpb.2021.11.003.

T. Stadler. Sampling-through-time in birth-death trees. *Journal of Theoretical Biology* **267**, 396–404, 2010. doi:10.1016/j.jtbi.2010.09.010.

T. Stadler. Sampling-through-time in birth-death trees. *Journal of Theoretical Biology* **267**, 396–404, 2010. doi:10.1016/j.jtbi.2010.09.010.

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

See Also

More example genealogy processes: [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```
simulate("LBDP",time=4) |> plot(points=TRUE)

simulate("LBDP",lambda=2,mu=1,psi=3,n0=1,time=1) |>
  simulate(time=10,lambda=1) |>
  plot()

simulate("LBDP",time=4) |>
  lineages() |>
  plot()
```

lineages	<i>Lineage-count function</i>
----------	-------------------------------

Description

Lineage-counts, saturations, and event-codes.

Usage

```
lineages(object, prune = TRUE, obscure = TRUE)

## S3 method for class 'gplin'
plot(x, ..., palette = scales::hue_pal(l = 30, h = c(220, 580)))
```

Arguments

<code>object</code>	gpsim object.
<code>prune</code>	logical; prune the genealogy?
<code>obscure</code>	logical; obscure the demes?
<code>x</code>	object of class 'gpger'
<code>...</code>	passed to theme .
<code>palette</code>	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.

Details

This function extracts from the specified genealogy several important time-varying quantities. These include:

lineages number of lineages through time

saturation the number of lineages emerging from the event

event_type an integer coding the type of event

If the genealogy has been obscured (the default), the number in the `lineages` returned is the total number of lineages present at the specified time and the saturation is the total saturation. If the genealogy has not been obscured (`obscure = FALSE`), the deme-specific data are returned. In this case, the `deme` column specifies the pertinent deme.

The event types are:

0 no event,

-1 a root,

1 a sample event,

2 a non-sample event,

3 the end of the time interval, which may or may not coincide with the latest tip of the genealogy.

`plot` applied to the data frame produced by `lineages` yields a lineage-through-time plot.

Value

A [tibble](#) containing information about the genealogy. See Details for specifics. The [tibble](#) returned by `lineages` has a [plot](#) method.

Examples

```
pal <- c("#000000", "#00274c", "#ffcb05")

simulate("SIIR", time=3, psi1=1, psi2=1) -> x
plot_grid(
  x |> plot(palette=pal),
  x |> lineages() |> plot(palette=pal),
```

```
x |> plot(obscure=FALSE,palette=pal),
x |> lineages(obscure=FALSE) |>
  plot(palette=pal,legend.position=c(0.8,0.9)),
  align="v",axis="b",
  ncol=2,byrow=FALSE
)
```

moran

The classical Moran model

Description

The Markov genealogy process induced by the classical Moran process, in which birth/death events occur at a constant rate and the population size remains constant.

Usage

```
runMoran(time, t0 = 0, mu = 1, psi = 1, n = 100)
```

```
continueMoran(object, time, mu = NA, psi = NA)
```

```
moran_exact(x, n = 100, mu = 1, psi = 1)
```

Arguments

time	final time
t0	initial time
mu	per capita event rate
psi	per capita sampling rate
n	population size
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format (i.e., an object that inherits from ‘gpgen’).

Details

`moran_exact` gives the exact log likelihood of a genealogy under the uniformly-sampled Moran process.

Value

`runMoran` and `continueMoran` return objects of class ‘gpsim’ with ‘model’ attribute “Moran”.

`moran_exact` returns the log likelihood of the genealogy.

References

P.A.P. Moran. Random processes in genetics. *Mathematical Proceedings of the Cambridge Philosophical Society* **54**, 60–71, 1958. doi:10.1017/s0305004100033193.

See Also

More example genealogy processes: [lbdp](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

newick	<i>Newick output</i>
--------	----------------------

Description

Extract a Newick-format description of a genealogy.

Usage

`newick(object, prune = TRUE, obscure = TRUE, extended = TRUE)`

Arguments

<code>object</code>	gpsim object.
<code>prune</code>	logical; prune the genealogy?
<code>obscure</code>	logical; obscure the demes?
<code>extended</code>	logical; if TRUE, an extended-Newick format is used. See Details.

Details

In the extended-Newick format, metadata tags of the form `[&&PhyloPOMP ...]` are inserted. These contain information regarding node-type and deme. See below for more information.

If `extended = FALSE`, then the string returned is in ordinary Newick format. Deme and node-type information is discarded. Every sample becomes a tip (inline samples lie on the ends of branches of zero length). Note that `extended = FALSE` implies both `prune = TRUE` and `obscure = TRUE`.

Value

A string in (possibly extended) Newick format.

Metadata tags

Metadata tags are of the form

`[&&PhyloPOMP type={node|sample|extant|root|migration|branch} deme=<integer>]`.

Types ‘branch’, ‘node’, ‘migration’, and ‘root’ are equivalent on input to [parse_newick](#); ‘type=node’ is used in output of [newick](#). Note that 0 is reserved for the “undeme”, i.e., unspecified or unknown deme.

Examples

```

simulate("SIIR",time=0.3,sigma12=1) -> x

x |> plot(prune=FALSE,obscure=FALSE)

x |> newick(prune=FALSE,obscure=FALSE)

x |> newick()

x |> newick(extended=FALSE)

x |>
  newick() |>
  parse_newick(t0=0)

x |>
  newick(prune=FALSE,obscure=FALSE) |>
  parse_newick(t0=0)

x |>
  newick(extended=FALSE) |>
  parse_newick(t0=0)

"" |> parse_newick() |> newick()

";" |> parse_newick() |> newick()

"((:1,:1):1,:1):1;" |> parse_newick() |> newick()

```

parse_newick	<i>parse a Newick string</i>
--------------	------------------------------

Description

Parses a Newick description and returns a binary version of the genealogy.

Usage

```
parse_newick(x, t0 = 0, tf = NA)
```

Arguments

x	character; the Newick description. See Details for specifics.
t0	numeric; the root time.
tf	numeric; the current or final time.

Details

parse_newick parses a string containing information in a (possibly extended) Newick format and returns a **phylopomp** genealogy.

The extension allows metadata (enclosed by square brackets). It ignores metadata unless they are specifically flagged as PhyloPOMP metadata, i.e.: [&&PhyloPOMP ...]. PhyloPOMP metadata can be used to encode the node type (i.e., type=sample, type=extant, type=node) and/or the deme (given as an integer, e.g., deme=1). See below for more information.

Value

An object of class “gpgen”.

Metadata tags

Metadata tags are of the form

```
[&&PhyloPOMP type={node|sample|extant|root|migration|branch} deme=<integer>].
```

Types ‘branch’, ‘node’, ‘migration’, and ‘root’ are equivalent on input to [parse_newick](#); ‘type=node’ is used in output of [newick](#). Note that 0 is reserved for the “undeme”, i.e., unspecified or unknown deme.

Examples

```
simulate("SIIR",time=0.3,sigma12=1) -> x

x |> plot(prune=FALSE,obscure=FALSE)

x |> newick(prune=FALSE,obscure=FALSE)

x |> newick()

x |> newick(extended=FALSE)

x |>
  newick() |>
  parse_newick(t0=0)

x |>
  newick(prune=FALSE,obscure=FALSE) |>
  parse_newick(t0=0)

x |>
  newick(extended=FALSE) |>
  parse_newick(t0=0)

"" |> parse_newick() |> newick()

";" |> parse_newick() |> newick()

"((:1,:1):1,:1):1;" |> parse_newick() |> newick()
```

phylopomp

Phylogenetics for POMP models

Description

Simulation and inference of Markov genealogy processes.

Author(s)

Aaron A. King, Qianying Lin

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:[10.48550/arxiv.2405.17032](https://doi.org/10.48550/arxiv.2405.17032).

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:[10.1016/j.tpb.2021.11.003](https://doi.org/10.1016/j.tpb.2021.11.003).

plot

Fancy tree plotter

Description

Plots a genealogical tree.

Usage

```
## S3 method for class 'gpngen'
plot(
  x,
  ...,
  prune = TRUE,
  obscure = TRUE,
  points = FALSE,
  ladderize = TRUE,
  palette = scales::hue_pal(l = 30, h = c(220, 580))
)
```

Arguments

x	object of class ‘gpngen’
...	plot passes extra arguments to theme .
prune	logical; prune the genealogy?
obscure	logical; obscure the demes?

points	logical; show nodes and tips?
ladderize	logical; ladderize?
palette	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.

Details

Tree-plotting in **phylopomp** depends on the **ggtree** package.

Value

A printable ggplot object.

References

G. Yu, D. K. Smith, H. Zhu, Y. Guan, T. T. Y. Lam. **ggtree**: an R package for visualization and annotation of phylogenetic trees with their covariates and other associated data. *Methods in Ecology and Evolution* **8**, 28–36, 2017. doi:[10.1111/2041210X.12628](https://doi.org/10.1111/2041210X.12628).

Examples

```
simulate("SEIR",Beta=2,sigma=2,gamma=1,psi=2,S0=1000,I0=5,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()

runSEIR(Beta=3,gamma=1,psi=2,S0=20,I0=5,R0=0,time=5,t0=-1) |>
  plot(points=TRUE,obscure=FALSE)

runSEIR(Beta=3,gamma=0.1,psi=0.2,S0=100,I0=5,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))

simulate("SEIR",sigma=1,omega=1,time=20,I0=4) |> plot(obscure=FALSE)

simulate("SEIR",sigma=1,omega=1,time=20,I0=4) |>
  lineages(obscure=FALSE) |>
  plot()
simulate("LBDP",time=4) |> plot(points=TRUE)

simulate("LBDP",lambda=2,mu=1,psi=3,n0=1,time=1) |>
  simulate(time=10,lambda=1) |>
  plot()

simulate("LBDP",time=4) |>
  lineages() |>
  plot()

runS2I2R2(
  time=30,
```

```

    iota2=0.01,Beta12=0.1,
    I1_0=0,I2_0=0,
    psi1=10,psi2=10,
    omega1=0.2,
    b2=0.02,d2=0.02
  ) |>
  freeze(seed=847110120) -> x

plot_grid(
  x |>
    plot(
      obscure=FALSE,
      palette=c("#000000ff","#440154ff","#21908cff","#ccccccff"),
      legend.position="inside",
      legend.position.inside=c(0.7,0.7)
    ),
  x |>
    lineages(obscure=FALSE) |>
    plot(
      palette=c("#000000ff","#440154ff","#21908cff","#ccccccff"),
      legend.position="none"
    ),
  ncol=1,
  align="hv",
  axis="b"
)
## Not run:

library(ggplot2)
library(phylopomp)
times <- seq(from=0,to=8,by=0.1)[-1]

png_files <- sprintf(
  file.path(tempdir(),"frame%05d.png"),
  seq_len(2*length(times))
)

pb <- utils::txtProgressBar(0,2*length(times),0,style=3)
x <- simulate("SIIR",time=0,Beta1=5,Beta2=10,gamma=1,omega=0.5,
  psi1=0.2,psi2=0.1,sigma12=1,sigma21=1,S0=200,I1_0=3,I2_0=2)

img <- 1
for (k in seq.int(from=1,to=length(times),by=1)) {
  x <- simulate(x,time=times[k])
  ggsave(
    filename=png_files[img],
    plot=plot(
      x, t0=0, time=max(times),
      points=FALSE, prune=FALSE, obscure=FALSE,
      palette=c("#ffcb05","#dddddd"),
      axis.line=element_line(color="white"),
      axis.ticks=element_line(color="white"),
      axis.text=element_blank(),

```

```

        plot.background=element_rect(fill=NA,color=NA),
        panel.background=element_rect(fill=NA,color=NA)
    ),
    device="png",dpi=300,
    height=2,width=3,units="in"
)
setTxtProgressBar(pb,img)
img <- img+1
}

for (k in seq.int(from=length(times),to=1,by=-1)) {
  x <- curtail(x,time=times[k])
  ggsave(
    filename=png_files[img],
    plot=plot(
      x, t0=0, time=max(times),
      points=FALSE, prune=FALSE, obscure=FALSE,
      palette=c("#ffcb05", "#dddddd"),
      axis.line=element_line(color="white"),
      axis.ticks=element_line(color="white"),
      axis.text=element_blank(),
      plot.background=element_rect(fill=NA,color=NA),
      panel.background=element_rect(fill=NA,color=NA)
    ),
    device="png",dpi=300,
    height=2,width=3,units="in"
  )
  setTxtProgressBar(pb,img)
  img <- img+1
}

library(gifski)
gif_file <- "movie1.gif"
gifski(png_files,gif_file,delay=0.08,loop=TRUE)
unlink(png_files)

## End(Not run)

```

reexports

Objects exported from other packages

Description

These objects are imported from other packages. Follow the links below to see their documentation.

cowplot [plot_grid](#)

foreach [%dopar%](#), [foreach](#), [registerDoSEQ](#)

grid [viewport](#)

```
pomp bake, freeze, stew
yaml as.yaml, read_yaml
```

s2i2r2

Two-host infection model with waning, immigration, and demography.

Description

The population is structured by infection progression and host species.

Usage

```
runS2I2R2(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueS2I2R2(
  object,
  time,
  Beta11 = NA,
  Beta12 = NA,
  Beta22 = NA,
  gamma1 = NA,
  gamma2 = NA,
  psi1 = NA,
```

```

psi2 = NA,
omega1 = NA,
omega2 = NA,
b1 = NA,
b2 = NA,
d1 = NA,
d2 = NA,
iota1 = NA,
iota2 = NA
)

```

Arguments

time	final time
t0	initial time
Beta11, Beta22	transmission rates within species 1 and 2, respectively
Beta12	transmission from species 2 to species 1
gamma1, gamma2	recovery rates for species 1 and 2, respectively
psi1, psi2	per capita sampling rates
omega1, omega2	rates of waning of immunity
b1, b2	per capita birth rates
d1, d2	per capita death rates
iota1, iota2	infection importation rates
S1_0, S2_0	initial sizes of susceptible populations
I1_0, I2_0	initial sizes of infected populations
R1_0, R2_0	initial sizes of immune populations
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object

Value

runS2I2R2 and continueS2I2R2 return objects of class ‘gpsim’ with ‘model’ attribute “S2I2R2”.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```

runS2I2R2(
  time=30,
  iota2=0.01, Beta12=0.1,
  I1_0=0, I2_0=0,
  psi1=10, psi2=10,
  omega1=0.2,

```

```

      b2=0.02,d2=0.02
    ) |>
      freeze(seed=847110120) -> x

plot_grid(
  x |>
    plot(
      obscure=FALSE,
      palette=c("#000000ff", "#440154ff", "#21908cff", "#ccccccff"),
      legend.position="inside",
      legend.position.inside=c(0.7,0.7)
    ),
  x |>
    lineages(obscure=FALSE) |>
    plot(
      palette=c("#000000ff", "#440154ff", "#21908cff", "#ccccccff"),
      legend.position="none"
    ),
  ncol=1,
  align="hv",
  axis="b"
)

```

seir

Classical susceptible-exposed-infected-recovered model

Description

The population is structured by infection progression.

Usage

```

runSEIR(
  time,
  t0 = 0,
  Beta = 4,
  sigma = 1,
  gamma = 1,
  psi = 1,
  omega = 0,
  S0 = 0.9,
  E0 = 0.05,
  I0 = 0.05,
  R0 = 0,
  pop = 100
)

```

```

runSEIRS(
  time,

```

```

    t0 = 0,
    Beta = 4,
    sigma = 1,
    gamma = 1,
    psi = 1,
    omega = 0,
    S0 = 0.9,
    E0 = 0.05,
    I0 = 0.05,
    R0 = 0,
    pop = 100
  )

  continueSEIR(
    object,
    time,
    Beta = NA,
    sigma = NA,
    gamma = NA,
    psi = NA,
    omega = NA
  )

  continueSEIRS(
    object,
    time,
    Beta = NA,
    sigma = NA,
    gamma = NA,
    psi = NA,
    omega = NA
  )

  seirs_pomp(x, Beta, sigma, gamma, psi, omega = 0, S0, E0, I0, R0, pop)

```

Arguments

time	final time
t0	initial time
Beta	transmission rate
sigma	progression rate
gamma	recovery rate
psi	per capita sampling rate
omega	rate of waning of immunity
S0, E0, I0, R0	initial conditions; non-negative numbers that specify the relative occupancies of the compartments at the initial time.

pop	host population size
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format.

Details

seirs_pomp constructs a ‘pomp’ object containing a given set of data and an SEIRS model.

Value

runSEIR and continueSEIR return objects of class ‘gpsim’ with ‘model’ attribute “SEIR”.

seirs_pomp returns a ‘pomp’ object.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:[10.48550/arxiv.2405.17032](https://doi.org/10.48550/arxiv.2405.17032).

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```
simulate("SEIR",Beta=2,sigma=2,gamma=1,psi=2,S0=1000,I0=5,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()
```

```
runSEIR(Beta=3,gamma=1,psi=2,S0=20,I0=5,R0=0,time=5,t0=-1) |>
  plot(points=TRUE,obscure=FALSE)
```

```
runSEIR(Beta=3,gamma=0.1,psi=0.2,S0=100,I0=5,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))
```

```
simulate("SEIR",sigma=1,omega=1,time=20,I0=4) |> plot(obscure=FALSE)
```

```
simulate("SEIR",sigma=1,omega=1,time=20,I0=4) |>
  lineages(obscure=FALSE) |>
  plot()
```

si2r

*Two-deme model of superspreading***Description**

Deme 2 consists of "superspreaders" who engender clusters of infection in "superspreading events".

Usage

```
runSI2R(
  time,
  t0 = 0,
  Beta = 5,
  mu = 5,
  gamma = 1,
  omega = 0,
  psi1 = 1,
  psi2 = 0,
  sigma12 = 1,
  sigma21 = 3,
  S0 = 500,
  I0 = 10,
  R0 = 0
)

continueSI2R(
  object,
  time,
  Beta = NA,
  mu = NA,
  gamma = NA,
  omega = NA,
  psi1 = NA,
  psi2 = NA,
  sigma12 = NA,
  sigma21 = NA
)
```

Arguments

time	final time
t0	initial time
Beta	transmission rate
mu	mean superspreading-event cluster size
gamma	recovery rate

omega	rate of waning of immunity
psi1, psi2	sampling rates for demes 1 and 2, respectively
sigma12, sigma21	movement rates from deme 1 to 2 and 2 to 1, respectively
S0	initial size of susceptible population
I0	initial size of I1 population (I2 = 0 at t = 0)
R0	initial size of recovered population
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object

Details

Superspreaders (deme 2) behave differently than ordinary infections: transmission events occur at the same rate (Beta), but at each event, a superspreader infects N individuals, where

$$N \sim 1 + \text{Geometric}(1/\mu).$$

Thus, assuming susceptibles are not limiting, the mean number of infections resulting from a superspreading event is μ and the variance in this number is $\mu^2 - \mu$. If susceptibles are limiting, i.e., if the number of susceptibles is not greater than N , then all remaining susceptibles are infected.

Value

runSI2R and continueSI2R return objects of class ‘gpsim’ with ‘model’ attribute “SI2R”.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```
simulate("SI2R",time=1) |>
  plot(obscure=FALSE)

runSI2R(Beta=10,S0=2000,time=1,psi1=0) |>
  simulate(time=2,psi1=1) |>
  plot(points=TRUE,obscure=FALSE)

simulate("SI2R",time=5) |>
  lineages() |>
  plot()

simulate("SI2R",time=2) |>
  diagram(m=30)

simulate("SI2R",time=20,omega=0.2,mu=20) -> x
plot_grid(
  x |> plot(obscure=FALSE),
  x |> lineages(obscure=FALSE) |> plot(),
```

```

ncol=1,
align="v",axis="b"
)

```

siir

Two-strain SIR model.

Description

Two distinct pathogen strains compete for susceptibles.

Usage

```

runSIIR(
  time,
  t0 = 0,
  Beta1 = 5,
  Beta2 = 5,
  gamma = 1,
  psi1 = 1,
  psi2 = 0,
  sigma12 = 0,
  sigma21 = 0,
  omega = 0,
  S0 = 500,
  I1_0 = 10,
  I2_0 = 10,
  R0 = 0
)

continueSIIR(
  object,
  time,
  Beta1 = NA,
  Beta2 = NA,
  gamma = NA,
  psi1 = NA,
  psi2 = NA,
  sigma12 = NA,
  sigma21 = NA,
  omega = NA
)

```

Arguments

time	final time
t0	initial time

Beta1, Beta2	transmission rates from each of the infectious classes.
gamma	recovery rate.
psi1, psi2	sampling rates.
sigma12, sigma21	movement rates from deme 1 to 2 and 2 to 1, respectively
omega	rate of loss of immunity
S0	initial size of susceptible population.
I1_0	initial size of I2 population.
I2_0	initial size of I2 population.
R0	initial size of recovered population.
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object

Value

runSIIR and continueSIIR return objects of class ‘gpsim’ with ‘model’ attribute “SIIR”.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```
simulate("SIIR", time=3, psi1=1, psi2=0) |>
  simulate(Beta1=2, gamma=2, time=10, psi1=10, psi2=1) |>
  plot()

runSIIR(Beta1=10, Beta2=8,
  S0=200, I1_0=10, I2_0=8, R0=0, time=0, t0=-1) |>
  simulate(psi1=10, time=2) |>
  plot(points=TRUE, obscure=FALSE)

simulate("SIIR", Beta1=2, Beta2=50, gamma=1, psi1=2,
  S0=300, I1_0=20, I2_0=2, time=5) |>
  lineages() |>
  plot()
```

simulate

simulate

Description

Simulate Markov genealogy processes

Usage

```
simulate(object, ...)

## Default S3 method:
simulate(object, ...)

## S3 method for class 'character'
simulate(object, time, ...)

## S3 method for class 'gpsim'
simulate(object, time, ...)
```

Arguments

<code>object</code>	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
<code>...</code>	additional arguments to the model-specific simulation functions
<code>time</code>	end timepoint of simulation

Details

When `object` is of class ‘gpsim’, i.e., the result of a genealogy-process simulation, `simulate` acts to continue the simulation to a later timepoint. Note that, one cannot change initial conditions or t_0 when continuing a simulation.

Value

An object of ‘gpsim’ class.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:[10.48550/arxiv.2405.17032](https://doi.org/10.48550/arxiv.2405.17032).

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:[10.1016/j.tpb.2021.11.003](https://doi.org/10.1016/j.tpb.2021.11.003).

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [sir](#), [strains](#), [twospecies](#), [twoundead](#)

sir*Classical susceptible-infected-recovered model*

Description

A single, unstructured population of hosts.

Usage

```
runSIR(  
  time,  
  t0 = 0,  
  Beta = 2,  
  gamma = 1,  
  psi = 1,  
  omega = 0,  
  S0 = 100,  
  I0 = 2,  
  R0 = 0,  
  pop = 102  
)
```

```
runSIRS(  
  time,  
  t0 = 0,  
  Beta = 2,  
  gamma = 1,  
  psi = 1,  
  omega = 0,  
  S0 = 100,  
  I0 = 2,  
  R0 = 0,  
  pop = 102  
)
```

```
continueSIR(object, time, Beta = NA, gamma = NA, psi = NA, omega = NA)
```

```
runSIRS(  
  time,  
  t0 = 0,  
  Beta = 2,  
  gamma = 1,  
  psi = 1,  
  omega = 0,  
  S0 = 100,  
  I0 = 2,  
  R0 = 0,
```

```

    pop = 102
  )

  continueSIRS(object, time, Beta = NA, gamma = NA, psi = NA, omega = NA)

  sir_pomp(x, Beta, gamma, psi, omega = 0, S0, I0, R0, pop)

  sirs_pomp(x, Beta, gamma, psi, omega = 0, S0, I0, R0, pop)

```

Arguments

time	final time
t0	initial time
Beta	transmission rate.
gamma	recovery rate.
psi	sampling rate.
omega	immunity waning rate
S0, I0, R0	initial conditions; non-negative numbers that specify the relative occupancies of the compartments at the initial time.
pop	host population size
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format (i.e., an object that inherits from ‘gpgen’).

Details

sir_pomp constructs a ‘pomp’ object containing a given set of data and a SIR model.

Value

runSIR and continueSIR return objects of class ‘gpsim’ with ‘model’ attribute “SIR”.

sir_pomp and sirs_pomp return ‘pomp’ objects.

References

- A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:[10.48550/arxiv.2405.17032](https://doi.org/10.48550/arxiv.2405.17032).
- A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:[10.1016/j.tpb.2021.11.003](https://doi.org/10.1016/j.tpb.2021.11.003).

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [strains](#), [twospecies](#), [twoundead](#)

Examples

```
simulate("SIR",Beta=2,gamma=1,psi=2,S0=1000,I0=5,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()

runSIR(Beta=3,gamma=1,psi=2,S0=20,I0=5,R0=0,time=5,t0=-1) |>
  plot(points=TRUE)

runSIR(Beta=3,gamma=0.1,psi=0.2,S0=100,I0=5,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))

simulate("SIRS",omega=1,time=20,I0=4) |> plot()
simulate("SIRS",omega=1,time=20,I0=4) |> lineages() |> plot()
```

strains

Three strains compete for a single susceptible pool.

Description

The three demes are three distinct pathogen strains that compete for susceptibles.

Usage

```
runStrains(
  time,
  t0 = 0,
  Beta1 = 5,
  Beta2 = 5,
  Beta3 = 5,
  gamma = 1,
  psi1 = 1,
  psi2 = 0,
  psi3 = 0,
  S0 = 1,
  I1_0 = 0.01,
  I2_0 = 0.01,
  I3_0 = 0.01,
  R0 = 0,
  pop = 10000
)

continueStrains(
  object,
  time,
  Beta1 = NA,
  Beta2 = NA,
```

```

    Beta3 = NA,
    gamma = NA,
    psi1 = NA,
    psi2 = NA,
    psi3 = NA
  )

  strains_pomp(
    x,
    Beta1,
    Beta2,
    Beta3,
    gamma,
    psi1,
    psi2,
    psi3,
    pop,
    S0,
    I1_0,
    I2_0,
    I3_0,
    R0
  )

```

Arguments

time	final time
t0	initial time
Beta1, Beta2, Beta3	transmission rate for strains 1, 2, 3, respectively
gamma	recovery rate
psi1, psi2, psi3	sampling rates
S0, I1_0, I2_0, I3_0, R0	initial conditions; non-negative numbers that specify the relative occupancies of the compartments at the initial time.
pop	host population size
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format (i.e., an object that inherits from ‘gpgen’).

Details

strains_pomp constructs a ‘pomp’ object containing a given set of data and the Strains model.

Value

runStrains and continueStrains return objects of class ‘gpsim’ with ‘model’ attribute “Strains”. strains_pomp returns a ‘pomp’ object.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [twospecies](#), [twoundead](#)

twospecies

Two-host infection model with waning, immigration, demography, and spillover. Hosts are culled upon sampling with a given probability.

Description

The population is structured by infection progression and host species.

Usage

```
runTwoSpecies(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta21 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  c1 = 1,
  c2 = 1,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueTwoSpecies(
  object,
  time,
```

```
Beta11 = NA,  
Beta12 = NA,  
Beta21 = NA,  
Beta22 = NA,  
gamma1 = NA,  
gamma2 = NA,  
psi1 = NA,  
psi2 = NA,  
c1 = NA,  
c2 = NA,  
omega1 = NA,  
omega2 = NA,  
b1 = NA,  
b2 = NA,  
d1 = NA,  
d2 = NA,  
iota1 = NA,  
iota2 = NA  
)  
  
twospecies_pomp(  
  x,  
  Beta11,  
  Beta12,  
  Beta21,  
  Beta22,  
  gamma1,  
  gamma2,  
  psi1,  
  psi2,  
  c1,  
  c2,  
  omega1,  
  omega2,  
  b1,  
  b2,  
  d1,  
  d2,  
  S1_0,  
  S2_0,  
  I1_0,  
  I2_0,  
  R1_0,  
  R2_0  
)
```

Arguments

time	final time
------	------------

t0	initial time
Beta11	transmission rate within species 1
Beta12	transmission from species 2 to species 1
Beta21	transmission from species 1 to species 2
Beta22	transmission rate within species 2
gamma1	species 1 recovery rate
gamma2	species 2 recovery rate
psi1	per capita sampling rate for species 1
psi2	per capita sampling rate for species 2
c1	probability that a sampled (positive) host of species 1 is culled
c2	probability that a sampled (positive) host of species 2 is culled
omega1	rate of waning of immunity for species 1
omega2	rate of waning of immunity for species 2
b1	per capita birth rate for species 1
b2	per capita birth rate for species 2
d1	per capita death rate for species 1
d2	per capita death rate for species 2
iota1	imported infections for species 1
iota2	imported infections for species 2
S1_0	initial size of species 1 susceptible population
S2_0	initial size of species 2 susceptible population
I1_0	initial size of species 1 infected population
I2_0	initial size of species 2 infected population
R1_0	initial size of species 1 immune population
R2_0	initial size of species 2 immune population
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object
x	genealogy in phylopomp format.

Details

twospecies_pomp constructs a ‘pomp’ object containing a given set of data and a TwoSpecies model. Note that, for the moment, twospecies_pomp assumes that there is no importation of infection into the populations (i.e., $iota1 = iota2 = 0$).

Value

runTwoSpecies and continueTwoSpecies return objects of class ‘gpsim’ with ‘model’ attribute “TwoSpecies”.

twospecies_pomp returns a ‘pomp’ object.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twoundead](#)

twoundead	<i>Two-host infection model with waning, immigration, demography, and spillover. Hosts are culled upon sampling with a given probability. This is identical to the TwoSpecies model with the exception that dead lineages are not pruned. Instead, they become ghosts.</i>
-----------	--

Description

The population is structured by infection progression and host species.

Usage

```
runTwoUndead(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta21 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  c1 = 1,
  c2 = 1,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueTwoUndead(
```

```

    object,
    time,
    Beta11 = NA,
    Beta12 = NA,
    Beta21 = NA,
    Beta22 = NA,
    gamma1 = NA,
    gamma2 = NA,
    psi1 = NA,
    psi2 = NA,
    c1 = NA,
    c2 = NA,
    omega1 = NA,
    omega2 = NA,
    b1 = NA,
    b2 = NA,
    d1 = NA,
    d2 = NA,
    iota1 = NA,
    iota2 = NA
  )

```

Arguments

time	final time
t0	initial time
Beta11	transmission rate within species 1
Beta12	transmission from species 2 to species 1
Beta21	transmission from species 1 to species 2
Beta22	transmission rate within species 2
gamma1	species 1 recovery rate
gamma2	species 2 recovery rate
psi1	per capita sampling rate for species 1
psi2	per capita sampling rate for species 2
c1	probability that a sampled (positive) host of species 1 is culled
c2	probability that a sampled (positive) host of species 2 is culled
omega1	rate of waning of immunity for species 1
omega2	rate of waning of immunity for species 2
b1	per capita birth rate for species 1
b2	per capita birth rate for species 2
d1	per capita death rate for species 1
d2	per capita death rate for species 2
iota1	imported infections for species 1

iota2	imported infections for species 2
S1_0	initial size of species 1 susceptible population
S2_0	initial size of species 2 susceptible population
I1_0	initial size of species 1 infected population
I2_0	initial size of species 2 infected population
R1_0	initial size of species 1 immune population
R2_0	initial size of species 2 immune population
object	either the name of the model to simulate <i>or</i> a previously computed ‘gpsim’ object

Value

runTwoUndead and continueTwoUndead return objects of class ‘gpsim’ with ‘model’ attribute “TwoUndead”.

See Also

More example genealogy processes: [lbdp](#), [moran](#), [s2i2r2](#), [seir](#), [si2r](#), [siir](#), [simulate\(\)](#), [sir](#), [strains](#), [twospecies](#)

yaml	<i>YAML output</i>
------	--------------------

Description

Human- and machine-readable description.

Usage

```
yaml(object)
```

Arguments

object gpsim object.

Value

A string in YAML format, with class “gpyaml”.

Examples

```
simulate("SIIR",time=1) |> yaml()
```

Index

* **Genealogy processes**

- lbdp, [10](#)
- moran, [13](#)
- s2i2r2, [21](#)
- seir, [23](#)
- si2r, [26](#)
- siir, [28](#)
- simulate, [29](#)
- sir, [31](#)
- strains, [33](#)
- twospecies, [35](#)
- twoundead, [38](#)

* **internals**

- genealogy diagram internals, [6](#)

* **internal**

- as.data.frame, [2](#)
- genealogy diagram internals, [6](#)
- getInfo, [8](#)
- reexports, [20](#)

+.gpgen (geneal_sum), [5](#)

%dopar% (reexports), [20](#)

%dopar%, [20](#)

as.data.frame, [2](#)

as.yaml, [21](#)

as.yaml (reexports), [20](#)

bake, [21](#)

bake (reexports), [20](#)

ballGrob (genealogy diagram internals),
[6](#)

continueLBDP (lbdp), [10](#)

continueMoran (moran), [13](#)

continueS2I2R2 (s2i2r2), [21](#)

continueSEIR (seir), [23](#)

continueSEIRS (seir), [23](#)

continueSI2R (si2r), [26](#)

continueSIIR (siir), [28](#)

continueSIR (sir), [31](#)

continueSIRS (sir), [31](#)

continueStrains (strains), [33](#)

continueTwoSpecies (twospecies), [35](#)

continueTwoUndead (twoundead), [38](#)

curtail, [2](#)

diagram, [3](#)

drawDetails.ballGrob (genealogy
diagram internals), [6](#)

drawDetails.resizingTextGrob
(genealogy diagram internals),
[6](#)

foreach, [20](#)

foreach (reexports), [20](#)

freeze, [21](#)

freeze (reexports), [20](#)

gendat, [4](#)

geneal, [5](#)

geneal_sum, [5](#)

genealogy diagram internals, [6](#)

genealogyGrob (genealogy diagram
internals), [6](#)

getInfo, [8](#)

LBDP (lbdp), [10](#)

lbdp, [10](#), [14](#), [22](#), [25](#), [27](#), [29](#), [30](#), [32](#), [35](#), [38](#), [40](#)

lbdp_exact (lbdp), [10](#)

lbdp_pomp (lbdp), [10](#)

lineages, [11](#)

Moran (moran), [13](#)

moran, [11](#), [13](#), [22](#), [25](#), [27](#), [29](#), [30](#), [32](#), [35](#), [38](#), [40](#)

moran_exact (moran), [13](#)

newick, [14](#), [14](#), [16](#)

nodeGrob (genealogy diagram internals),
[6](#)

parse_newick, [14](#), [15](#), [16](#)

phylopomp, 17
 phylopomp, package (phylopomp), 17
 phylopomp-package (phylopomp), 17
 plot, 12, 17
 plot.gplin (lineages), 11
 plot_grid, 20
 plot_grid (reexports), 20
 pocketGrob (genealogy diagram
 internals), 6
 postDrawDetails.ballGrob (genealogy
 diagram internals), 6
 postDrawDetails.resizingTextGrob
 (genealogy diagram internals),
 6
 preDrawDetails.ballGrob (genealogy
 diagram internals), 6
 preDrawDetails.resizingTextGrob
 (genealogy diagram internals),
 6
 print.gpdia (diagram), 3
 print.gpgen (genealogy diagram
 internals), 6
 print.gpsim (genealogy diagram
 internals), 6
 print.gpyaml (genealogy diagram
 internals), 6

 read_yaml, 21
 read_yaml (reexports), 20
 reexports, 20
 registerDoSEQ, 20
 registerDoSEQ (reexports), 20
 resizingTextGrob (genealogy diagram
 internals), 6
 runLBDP (lbdp), 10
 runMoran (moran), 13
 runS2I2R2 (s2i2r2), 21
 runSEIR (seir), 23
 runSEIRS (seir), 23
 runSI2R (si2r), 26
 runSIIR (siir), 28
 runSIR (sir), 31
 runSIRS (sir), 31
 runStrains (strains), 33
 runTwoSpecies (twospecies), 35
 runTwoUndead (twoundead), 38

 S2I2R2 (s2i2r2), 21
 s2i2r2, 11, 14, 21, 25, 27, 29, 30, 32, 35, 38,
 40
 SEIR (seir), 23
 seir, 11, 14, 22, 23, 27, 29, 30, 32, 35, 38, 40
 seirs_pomp (seir), 23
 SI2R (si2r), 26
 si2r, 11, 14, 22, 25, 26, 29, 30, 32, 35, 38, 40
 SIIR (siir), 28
 siir, 11, 14, 22, 25, 27, 28, 30, 32, 35, 38, 40
 simulate, 11, 14, 22, 25, 27, 29, 29, 32, 35,
 38, 40
 SIR (sir), 31
 sir, 11, 14, 22, 25, 27, 29, 30, 31, 35, 38, 40
 sir_pomp (sir), 31
 SIRS (sir), 31
 sirs_pomp (sir), 31
 stew, 21
 stew (reexports), 20
 Strains (strains), 33
 strains, 11, 14, 22, 25, 27, 29, 30, 32, 33, 38,
 40
 strains_pomp (strains), 33

 textGrob, 7
 theme, 12, 17
 tibble, 9, 12
 TwoSpecies (twospecies), 35
 twospecies, 11, 14, 22, 25, 27, 29, 30, 32, 35,
 35, 40
 twospecies_pomp (twospecies), 35
 TwoUndead (twoundead), 38
 twoundead, 11, 14, 22, 25, 27, 29, 30, 32, 35,
 38, 38

 viewport, 20
 viewport (reexports), 20

 yaml, 40